Separability, Tensor Products, and Structure-Aware Discretizations

Yao Zhang

October 13, 2026

Tencent Meeting ID: 596-9863-2355
Wednesdays, 13:30–15:30, beginning October 14, 2026

Abstract

This article develops a unified account of how structural properties of high-dimensional continuous models can be inherited by their finite-dimensional discretizations. Coordinate separability leads to tensor-product operators and, after bases and an ordering convention are chosen, to Kronecker products or sums of Kronecker products. Translation invariance leads to Toeplitz or circulant structure, while local support often produces sparsity. The framework is illustrated through Fourier transforms, wavelets, B-splines, radial basis functions, and convolution operators, with particular emphasis on the continuous–discrete transition. Sampling and quadrature, refinement equations, Galerkin projection, collocation, and kernel evaluation are treated as distinct discretization mechanisms. The resulting perspective separates the origin of a structure from its matrix representation and from the algorithm used to exploit it.

Keywords. Separability; tensor products; Kronecker products; structure-aware discretization; Fourier transforms; wavelets; B-splines; radial basis functions; Toeplitz matrices; convolution.

WORK IN PROGRESS — NOT FOR DISTRIBUTION

Contents

1 Introduction and Organizing Principle **1**

2 Notation and Indexing Conventions **2**

3 Tensor-Product Structure **3**

4 Kronecker Representations of Tensor-Product Operators **6**

5 Discretization Mechanisms **10**

6 Fourier Analysis: Separability, the DFT, and the FFT **12**

6.1 From Periodic Fourier Coefficients to the DFT 12

6.2 Kronecker Representation of the Multidimensional DFT 13

7 Wavelets: Multiresolution, Tensor Products, and the FWT **15**

7.1 Wavelet Matrix Representation versus Filter-Bank Implementation 17

8 B-Splines: Tensor-Product Approximation Spaces **18**

9 Radial Basis Functions: Radiality versus Separability **21**

10 Convolution: Transform Representations and Structured Matrices **23**

11 Summary **27**

WORK IN PROGRESS — NOT FOR DISTRIBUTION

1 Introduction and Organizing Principle

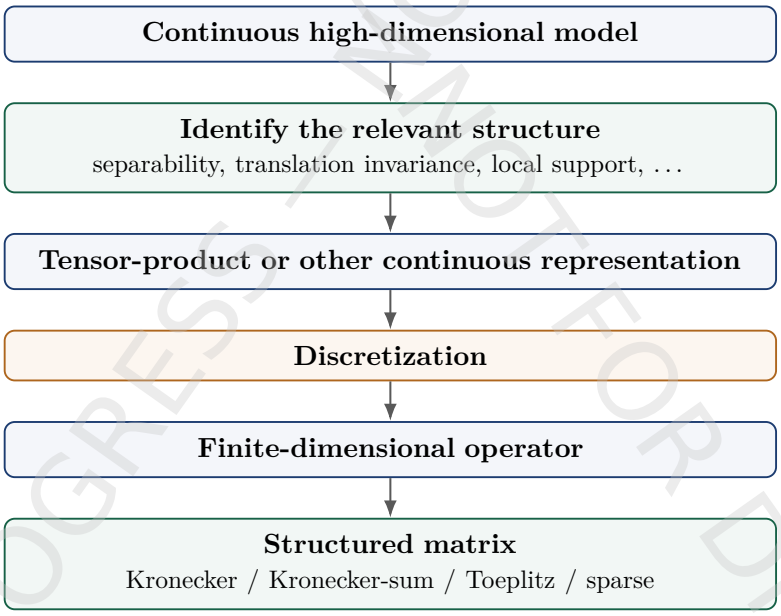
High dimensionality by itself does not imply tensor-product structure. The decisive question is whether the mathematical object under consideration separates with respect to its coordinate variables. For example, a kernel of the form

$$K(\boldsymbol{y}, \boldsymbol{x}) = \prod_{j=1}^{d} K_j(y_j, x_j) \tag{1.1}$$

separates coordinatewise. On a Cartesian product domain equipped with a product measure, this factorization is the structural reason that the associated multidimensional integral operator can be assembled from one-dimensional operators by tensor products.

The phrase *structure-aware discretization* is used here in a broader sense than *Kronecker product*. Different continuous properties produce different finite-dimensional structures. Separability produces tensor-product operators and, after bases and an ordering convention are chosen, Kronecker products or sums of Kronecker products. Translation invariance produces Toeplitz or circulant structure after suitable sampling. Local support, as in B-splines and compactly supported wavelets, often produces sparsity. Several of these properties may occur simultaneously, and a discretization may preserve some of them while modifying or destroying others.

Accordingly, the framework of this article is better represented by



Within this broader diagram, the principal thread studied here is



The distinction matters. A Toeplitz matrix is not Toeplitz merely because a kernel is separable; Toeplitz structure comes from dependence on a coordinate difference such as $x - y$. Likewise, sparsity is not a consequence of separability alone.

The continuous–discrete passage is also problem dependent. Sampling creates finite data, but it does not by itself define a discrete operator. Quadrature approximates integrals by weighted sums; Galerkin or Petrov–Galerkin projection produces matrices from trial and test spaces;

WORK IN PROGRESS — NOT FOR DISTRIBUTION

collocation enforces equations at selected points; and wavelet filter banks arise from refinement equations. Boundary conditions and boundary extensions are equally important because they determine, for example, whether a finite convolution matrix is Toeplitz or circulant. The purpose of the following sections is therefore not merely to list structured formulas, but to derive each formula from the continuous property and discretization mechanism that actually produces it.

2 Notation and Indexing Conventions

Only conventions used repeatedly throughout the article are fixed here; operation-specific notation is introduced where it first becomes necessary. Let $d \in \mathbb{N}$ be the number of coordinate directions. For coordinate j, let

$$N_j \in \mathbb{N}, \qquad \mathcal{I}_{N_j} = \{0, \dots, N_j - 1\}, \tag{2.1}$$

and define

$$\boldsymbol{N} = (N_1, \dots, N_d), \qquad N_{\mathrm{tot}} = \prod_{j=1}^{d} N_j. \tag{2.2}$$

Continuous variables are written as

$$\boldsymbol{x} = (x_1, \dots, x_d), \qquad \boldsymbol{y} = (y_1, \dots, y_d), \qquad \boldsymbol{\xi} = (\xi_1, \dots, \xi_d) \in \mathbb{R}^d. \tag{2.3}$$

A *multi-index*

$$\boldsymbol{n} = (n_1, \dots, n_d) \in \mathcal{I}_{\boldsymbol{N}} := \mathcal{I}_{N_1} \times \cdots \times \mathcal{I}_{N_d} \tag{2.4}$$

identifies one entry of an $N_1 \times \cdots \times N_d$ array. Thus each coordinate index is zero-based:

$$n_j \in \{0, \dots, N_j - 1\}. \tag{2.5}$$

The same convention is used for other discrete multi-indices, such as $\boldsymbol{k} = (k_1, \dots, k_d)$.

A scalar field sampled on a Cartesian grid is stored as an order-d scalar-valued array. To distinguish array shape from Cartesian products and tensor products of vector spaces, define

$$\mathcal{A}_{\boldsymbol{N}} := \mathbb{C}^{\mathcal{I}_{\boldsymbol{N}}} = \{X : \mathcal{I}_{\boldsymbol{N}} \to \mathbb{C}\}. \tag{2.6}$$

Thus

$$X = (x_{\boldsymbol{n}})_{\boldsymbol{n} \in \mathcal{I}_{\boldsymbol{N}}} \in \mathcal{A}_{\boldsymbol{N}}, \qquad x_{\boldsymbol{n}} := X(\boldsymbol{n}) \in \mathbb{C}. \tag{2.7}$$

Here $\boldsymbol{n} = (n_1, \dots, n_d)$ is a multidimensional index, whereas $x_{\boldsymbol{n}}$ is the scalar value stored at that index. Moreover,

$$\dim \mathcal{A}_{\boldsymbol{N}} = |\mathcal{I}_{\boldsymbol{N}}| = N_{\mathrm{tot}} = \prod_{j=1}^{d} N_j. \tag{2.8}$$

The common notation $\mathbb{C}^{N_1 \times \cdots \times N_d}$ may describe the *shape* of a multidimensional array, but it is avoided as the formal array-space notation below because it can be confused with Cartesian products.

WORK IN PROGRESS / NOT FOR DISTRIBUTION

Vectorization is the linear isomorphism

$$\mathrm{vec} : \mathcal{A}_{\boldsymbol{N}} \longrightarrow \mathbb{C}^{N_{\mathrm{tot}}}, \qquad X \longmapsto \mathrm{vec}(X). \tag{2.9}$$

Throughout the article, the first coordinate varies fastest. Define the zero-based linear index

$$\ell_{\boldsymbol{N}}(\boldsymbol{n}) = n_1 + N_1 n_2 + \cdots + \left(\prod_{r=1}^{d-1} N_r\right) n_d, \tag{2.10}$$

so that

$$0 \le \ell_{\boldsymbol{N}}(\boldsymbol{n}) \le N_{\mathrm{tot}} - 1. \tag{2.11}$$

Because mathematical vector components are conventionally numbered from 1, the scalar array entry indexed by $\boldsymbol{n}$ is placed in the scalar vector component

$$[\mathrm{vec}(X)]_{\ell_{\boldsymbol{N}}(\boldsymbol{n})+1} = x_{\boldsymbol{n}}, \qquad \boldsymbol{n} \in \mathcal{I}_{\boldsymbol{N}}. \tag{2.12}$$

Both sides of (2.12) belong to $\mathbb{C}$: the left-hand side is one scalar component of $\mathrm{vec}(X)$, whereas the right-hand side is one scalar entry of the multidimensional array X.

Equations (2.10)–(2.12) therefore use zero-based indexing for the multidimensional grid and for the linear index, while retaining the usual one-based notation for components of a mathematical column vector. In two dimensions this is column-major vectorization.

The ordering convention does not create tensor-product structure; it only determines how a multidimensional operator is written as a matrix after vectorization. Section 4 derives the resulting Kronecker factor order explicitly.

3 Tensor-Product Structure

Tensor products provide the natural language for combining coordinatewise vector spaces while retaining their factor structure. They should not be confused with Cartesian products of vector spaces, which represent a different algebraic construction. Let V_j be the vector or function space associated with coordinate j. The multidimensional space is formed from

$$V_1 \otimes \cdots \otimes V_d. \tag{3.1}$$

For two spaces V and W, the tensor product $V \otimes W$ is generated by elementary tensors $v \otimes w$ subject to bilinearity:

$$(v_1 + v_2) \otimes w = v_1 \otimes w + v_2 \otimes w, \qquad v \otimes (w_1 + w_2) = v \otimes w_1 + v \otimes w_2, \tag{3.2}$$

and

$$(\alpha v) \otimes w = v \otimes (\alpha w) = \alpha(v \otimes w). \tag{3.3}$$

A general tensor is a finite linear combination of elementary tensors. For d factors,

$$u = \sum_{r=1}^{R} v_1^{(r)} \otimes \cdots \otimes v_d^{(r)}. \tag{3.4}$$

WORK IN PROGRESS / NOT FOR DISTRIBUTION

The tensor product is associative up to the canonical identification, so parentheses in iterated products are normally omitted.

If $\dim V_j = N_j$ for $j = 1, \dots, d$, then the tensor products of basis vectors from the factor spaces form a basis of the tensor-product space. Hence

$$\dim(V_1 \otimes \cdots \otimes V_d) = \prod_{j=1}^{d} N_j = N_{\text{tot}}. \tag{3.5}$$

After bases are chosen, an elementary tensor of two coordinate vectors $\boldsymbol{a} \in \mathbb{C}^m$ and $\boldsymbol{b} \in \mathbb{C}^n$ has tensor-product coordinates $a_i b_j$. If these coordinates are reshaped as an $m \times n$ array, the result is the outer product

$$\boldsymbol{a}\boldsymbol{b}^T, \qquad (\boldsymbol{a}\boldsymbol{b}^T)_{ij} = a_i b_j. \tag{3.6}$$

Under the first-coordinate-fastest vectorization convention used below,

$$\operatorname{vec}(\boldsymbol{a}\boldsymbol{b}^T) = \boldsymbol{b} \otimes \boldsymbol{a}, \tag{3.7}$$

where the right-hand side is the usual Kronecker product of coordinate vectors. This reshaping convention should be distinguished from the abstract tensor product itself and from the Kronecker product of matrices, introduced in Section 4 as the matrix representation of tensor-product linear maps.

For function spaces on a Cartesian product domain, elementary tensors have a concrete product realization. If $f_j : X_j \to \mathbb{C}$, the bilinear (and, in higher dimensions, multilinear) product map induces the canonical identification

$$(f_1 \otimes \cdots \otimes f_d)(x_1, \dots, x_d) = \prod_{j=1}^{d} f_j(x_j). \tag{3.8}$$

Thus a product of one-dimensional functions is not merely analogous to an elementary tensor; it is its standard function-space realization. The exact functional-analytic completion depends on the spaces under consideration (for example, Hilbert tensor products in L^2 settings), but the elementary tensor formula above is the structural fact used throughout this article.

Consequently, a coordinate-separable function

$$f(x_1, \dots, x_d) = \prod_{j=1}^{d} f_j(x_j) \tag{3.9}$$

can be written as

$$f = f_1 \otimes \cdots \otimes f_d. \tag{3.10}$$

A general element of the algebraic tensor-product space need not have rank one. By definition, however, every

$$f \in V_1 \otimes \cdots \otimes V_d \tag{3.11}$$

can be written as a finite sum of elementary tensors:

WORK IN PROGRESS — NOT FOR DISTRIBUTION

$$f = \sum_{r=1}^{R} f_1^{(r)} \otimes \cdots \otimes f_d^{(r)}. \tag{3.12}$$

Under the corresponding function-space realization, this identity becomes

$$f(\boldsymbol{x}) = \sum_{r=1}^{R} f_1^{(r)}(x_1) \cdots f_d^{(r)}(x_d) = \sum_{r=1}^{R} \prod_{j=1}^{d} f_j^{(r)}(x_j). \tag{3.13}$$

Thus the equality above is an exact representation of an element of the algebraic tensor product; the integer R is simply the number of elementary tensor terms in the chosen representation and need not be minimal.

In numerical applications, or when one works in a completed tensor-product function space, a finite sum of this form may instead be used as a truncated finite-rank approximation to a more general function. In that distinct setting it is appropriate to write

$$f(\boldsymbol{x}) \approx \sum_{r=1}^{R} f_1^{(r)}(x_1) \cdots f_d^{(r)}(x_d), \tag{3.14}$$

where the approximation sign indicates a nonzero truncation or approximation error. This distinction separates the exact algebraic definition of a tensor product from its use in low-rank numerical approximation.

The same construction extends from vectors and functions to linear operators. Let

$$A_j : V_j \to W_j, \qquad j = 1, \ldots, d. \tag{3.15}$$

The tensor-product operator

$$A_1 \otimes \cdots \otimes A_d : V_1 \otimes \cdots \otimes V_d \longrightarrow W_1 \otimes \cdots \otimes W_d \tag{3.16}$$

is defined first on elementary tensors by

$$(A_1 \otimes \cdots \otimes A_d)(v_1 \otimes \cdots \otimes v_d) = (A_1 v_1) \otimes \cdots \otimes (A_d v_d), \tag{3.17}$$

and then extended linearly.

Now consider an integral operator on a Cartesian product domain $\Omega = \Omega_1 \times \cdots \times \Omega_d$ with product measure,

$$(Tf)(\boldsymbol{y}) = \int_{\Omega} K(\boldsymbol{y}, \boldsymbol{x}) f(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}. \tag{3.18}$$

Assume that the integrand satisfies hypotheses that justify repeated integration (for example, absolute integrability so that Fubini's theorem applies), and suppose

$$K(\boldsymbol{y}, \boldsymbol{x}) = \prod_{j=1}^{d} K_j(y_j, x_j). \tag{3.19}$$

For an elementary tensor

WORK IN PROGRESS / NOT FOR DISTRIBUTION

$$f(\boldsymbol{x}) = (f_1 \otimes \cdots \otimes f_d)(\boldsymbol{x}) = \prod_{j=1}^{d} f_j(x_j), \tag{3.20}$$

Fubini's theorem gives

$$\begin{aligned}(Tf)(\boldsymbol{y}) &= \int \prod_{j=1}^{d} K_j(y_j, x_j) f_j(x_j)\ \mathrm{d}\boldsymbol{x} \\ &= \prod_{j=1}^{d} \left[\int K_j(y_j, x_j) f_j(x_j)\ \mathrm{d}x_j \right].\end{aligned} \tag{3.21}$$

If

$$(T_j f_j)(y_j) = \int_{\Omega_j} K_j(y_j, x_j) f_j(x_j)\ \mathrm{d}x_j, \tag{3.22}$$

then

$$T(f_1 \otimes \cdots \otimes f_d) = (T_1 f_1) \otimes \cdots \otimes (T_d f_d). \tag{3.23}$$

This is exactly the defining action of a tensor-product operator. Therefore, under the usual functional-analytic assumptions,

$$T = T_1 \otimes \cdots \otimes T_d. \tag{3.24}$$

More generally, if the kernel has finite separation rank,

$$K(\boldsymbol{y}, \boldsymbol{x}) = \sum_{r=1}^{R} \prod_{j=1}^{d} K_j^{(r)}(y_j, x_j), \tag{3.25}$$

then linearity gives

$$T = \sum_{r=1}^{R} T_1^{(r)} \otimes \cdots \otimes T_d^{(r)}, \tag{3.26}$$

provided the same interchange-of-integration assumptions hold termwise. This observation is the continuous precursor of sums of Kronecker products in finite-dimensional discretizations.

This establishes the first central implication:

separable kernel $\Longrightarrow$ tensor-product operator.

4 Kronecker Representations of Tensor-Product Operators

After finite-dimensional bases are chosen, tensor-product operators acquire ordinary matrix representations. From this point onward, symbols such as A and B denote matrices, whereas coordinate representatives of individual vectors are understood as column vectors. Let

WORK IN PROGRESS — NOT FOR DISTRIBUTION

$$A = [a_{ij}] \in \mathbb{C}^{m\times n}, \qquad B = [b_{rs}] \in \mathbb{C}^{p\times q}. \tag{4.1}$$

Thus A maps an n-component coordinate column vector to an m-component one, and B maps a q-component coordinate column vector to a p-component one. Their Kronecker product is

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix} \in \mathbb{C}^{mp\times nq}. \tag{4.2}$$

Using paired row and column labels for clarity, its entries satisfy

$$(A \otimes B)_{(i,r),(j,s)} = a_{ij}b_{rs}, \qquad 1 \le i \le m,\ 1 \le j \le n,\ 1 \le r \le p,\ 1 \le s \le q. \tag{4.3}$$

After a specific linear ordering of the index pairs is chosen, this is an ordinary $mp \times nq$ matrix.

The same symbol $\otimes$ is used because, in tensor-product bases, the Kronecker product is precisely the matrix representation of the tensor product of finite-dimensional linear maps; standard algebraic identities and computational uses are reviewed, for example, by Van Loan [5].

For conformable factors,

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD), \qquad (A \otimes B)^* = A^* \otimes B^*. \tag{4.4}$$

If A and B are square and nonsingular, then

$$(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}. \tag{4.5}$$

These identities explain why factorizations, inverses, and matrix–vector products can often be carried out through the lower-dimensional factors rather than through the assembled Kronecker matrix.

The two-factor definition extends recursively to any number of coordinate directions. If

$$A_j \in \mathbb{C}^{M_j\times N_j}, \qquad j = 1, \dots, d, \tag{4.6}$$

then

$$A_d \otimes \cdots \otimes A_1 \tag{4.7}$$

denotes the repeated Kronecker product. For example,

$$A_3 \otimes A_2 \otimes A_1 = (A_3 \otimes A_2) \otimes A_1, \tag{4.8}$$

with dimensions

$$(M_3M_2M_1) \times (N_3N_2N_1). \tag{4.9}$$

In general,

WORK IN PROGRESS — NOT FOR DISTRIBUTION

$$A_d \otimes \cdots \otimes A_1 \in \mathbb{C}^{M_{\mathrm{tot}} \times N_{\mathrm{tot}}} \tag{4.10}$$

where

$$M_{\mathrm{tot}} = \prod_{j=1}^{d} M_j, \qquad N_{\mathrm{tot}} = \prod_{j=1}^{d} N_j. \tag{4.11}$$

At this point it is important to separate two ideas. The tensor-product structure is already present in the coordinatewise operator. Vectorization is not needed to create that structure, nor is it required to compute a multidimensional transform. It is needed only when we want to express the entire multidimensional action in the ordinary matrix–vector form

$$\boldsymbol{y} = A_{\mathrm{full}} \boldsymbol{x}. \tag{4.12}$$

The order-d input and output arrays must then be represented by the single coordinate vectors $\mathrm{vec}(X)$ and $\mathrm{vec}(Y)$. The chosen vectorization order determines the row and column numbering of A_{full}, and hence the displayed order of its Kronecker factors.

To see this directly in arbitrary dimension, let the order-d arrays be

$$X = (x_{\boldsymbol{n}})_{\boldsymbol{n} \in \mathcal{I}_{\boldsymbol{N}}} \in \mathcal{A}_{\boldsymbol{N}}, \qquad Y = (y_{\boldsymbol{m}})_{\boldsymbol{m} \in \mathcal{I}_{\boldsymbol{M}}} \in \mathcal{A}_{\boldsymbol{M}}, \tag{4.13}$$

where

$$\boldsymbol{n} = (n_1, \ldots, n_d), \qquad \boldsymbol{m} = (m_1, \ldots, m_d). \tag{4.14}$$

Suppose that the j-th coordinate is acted on by $A_j \in \mathbb{C}^{M_j \times N_j}$. The coordinatewise action is

$$y_{m_1,\ldots,m_d} = \sum_{n_1=0}^{N_1-1} \cdots \sum_{n_d=0}^{N_d-1} \left[\prod_{j=1}^{d} (A_j)_{m_j+1, n_j+1} \right] x_{n_1,\ldots,n_d}. \tag{4.15}$$

This formula already displays the separable product structure: the coefficient coupling the multi-indices $\boldsymbol{m}$ and $\boldsymbol{n}$ is a product of one-dimensional matrix entries.

For the input, use the first-coordinate-fastest index map from Section 2:

$$\ell_{\boldsymbol{N}}(\boldsymbol{n}) = n_1 + N_1 n_2 + \cdots + \left(\prod_{r=1}^{d-1} N_r \right) n_d. \tag{4.16}$$

For the output size vector $\boldsymbol{M} = (M_1, \ldots, M_d)$, define analogously

$$\ell_{\boldsymbol{M}}(\boldsymbol{m}) = m_1 + M_1 m_2 + \cdots + \left(\prod_{r=1}^{d-1} M_r \right) m_d. \tag{4.17}$$

Then

$$\mathrm{vec}(X) \in \mathbb{C}^{N_{\mathrm{tot}} \times 1}, \qquad \mathrm{vec}(Y) \in \mathbb{C}^{M_{\mathrm{tot}} \times 1}, \tag{4.18}$$

WORK IN PROGRESS — NOT FOR DISTRIBUTION

with

$$[\mathrm{vec}(X)]_{\ell_{\boldsymbol{N}}(\boldsymbol{n})+1} = x_{\boldsymbol{n}}, \qquad [\mathrm{vec}(Y)]_{\ell_{\boldsymbol{M}}(\boldsymbol{m})+1} = y_{\boldsymbol{m}}. \tag{4.19}$$

Under these index maps, the entries of the repeated Kronecker product satisfy

$$[A_d \otimes \cdots \otimes A_1]_{\ell_{\boldsymbol{M}}(\boldsymbol{m})+1,\, \ell_{\boldsymbol{N}}(\boldsymbol{n})+1} = \prod_{j=1}^{d} (A_j)_{m_j+1,n_j+1}. \tag{4.20}$$

Consequently, the $(\ell_{\boldsymbol{M}}(\boldsymbol{m})+1)$-st component of the ordinary matrix–vector product is

$$\begin{aligned} &[(A_d \otimes \cdots \otimes A_1)\,\mathrm{vec}(X)]_{\ell_{\boldsymbol{M}}(\boldsymbol{m})+1} \\ &\quad = \sum_{n_1=0}^{N_1-1} \cdots \sum_{n_d=0}^{N_d-1} \left[\prod_{j=1}^{d} (A_j)_{m_j+1,n_j+1}\right] x_{n_1,\ldots,n_d} \\ &\quad = y_{m_1,\ldots,m_d} = [\mathrm{vec}(Y)]_{\ell_{\boldsymbol{M}}(\boldsymbol{m})+1}. \end{aligned} \tag{4.21}$$

Since this holds for every output multi-index $\boldsymbol{m}$,

$$\mathrm{vec}(Y) = (A_d \otimes \cdots \otimes A_1)\,\mathrm{vec}(X). \tag{4.22}$$

Thus vectorization converts the multi-index relation into ordinary matrix–vector algebra, while the Kronecker product preserves the original coordinatewise product structure.

The general Kronecker formula and the two-dimensional matrix formula should be understood as two coordinate representations of the same underlying coordinatewise tensor-product action. Setting $d = 2$, the order-2 arrays are ordinary matrices, so

$$X \in \mathbb{C}^{N_1 \times N_2}, \qquad Y \in \mathbb{C}^{M_1 \times M_2}, \qquad A_1 \in \mathbb{C}^{M_1 \times N_1}, \qquad A_2 \in \mathbb{C}^{M_2 \times N_2}, \tag{4.23}$$

and the general result reduces to

$$\boxed{\mathrm{vec}(Y) = (A_2 \otimes A_1)\,\mathrm{vec}(X).}$$

Under the same first-coordinate-fastest convention, the corresponding component relation is

$$y_{m_1,m_2} = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} (A_1)_{m_1+1,n_1+1} (A_2)_{m_2+1,n_2+1} x_{n_1,n_2}. \tag{4.24}$$

On the other hand, the (m_1, m_2)-entry of the ordinary matrix product $A_1 X A_2^T$ is

$$\begin{aligned} [A_1 X A_2^T]_{m_1+1,m_2+1} &= \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} (A_1)_{m_1+1,n_1+1} x_{n_1,n_2} (A_2^T)_{n_2+1,m_2+1} \\ &= \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} (A_1)_{m_1+1,n_1+1} (A_2)_{m_2+1,n_2+1} x_{n_1,n_2} \\ &= y_{m_1,m_2}. \end{aligned} \tag{4.25}$$

Independently of vectorization, the same two-dimensional coordinatewise action has the direct matrix representation

WORK IN PROGRESS — NOT FOR DISTRIBUTION

$$Y = A_1 X A_2^T \in \mathbb{C}^{M_1 \times M_2}.$$

Thus

$$Y = A_1 X A_2^T \qquad \text{and} \qquad \operatorname{vec}(Y) = (A_2 \otimes A_1)\operatorname{vec}(X) \tag{4.26}$$

are two parallel representations of the same coordinatewise tensor-product action. The direct matrix identity is not derived from vectorization, and vectorization is not required to define the two-dimensional action. Both formulas express the same componentwise relation in different coordinate formats. The factor A_1 appears rightmost in $A_2 \otimes A_1$ solely because the first coordinate varies fastest under the chosen vectorization convention.

A different vectorization order changes the displayed ordering of the Kronecker factors. The underlying tensor-product operator would be unchanged; only its coordinate representation would change, with the different matrix forms related by the appropriate permutation matrices.

Combining the continuous and finite-dimensional levels gives the complete passage from separability to a Kronecker matrix representation:

$$\begin{gathered}
K(\boldsymbol{y}, \boldsymbol{x}) = \textstyle\prod_{j=1}^{d} K_j(y_j, x_j) \\
\Downarrow \\
T = T_1 \otimes \cdots \otimes T_d \qquad \text{tensor-product operator} \\
\Downarrow \quad \text{choose finite-dimensional bases} \\
A_j = \text{matrix of } T_j \\
\Downarrow \quad \text{represent multidimensional arrays by coordinate vectors} \\
X \mapsto \operatorname{vec}(X), \qquad Y \mapsto \operatorname{vec}(Y) \\
\Downarrow \quad \text{first-coordinate-fastest ordering} \\
\operatorname{vec}(Y) = \underbrace{(A_d \otimes \cdots \otimes A_1)}_{A_{\text{full}} \in \mathbb{C}^{M_{\text{tot}} \times N_{\text{tot}}}} \operatorname{vec}(X).
\end{gathered}$$

This makes the role of vectorization explicit: separability produces the tensor-product operator, whereas vectorization supplies the coordinate representation in which that operator becomes a Kronecker matrix. In this sense,

$$\text{Separability} \Longrightarrow \text{Tensor Products} \Longrightarrow \text{structure-aware discretizations.}$$

5 Discretization Mechanisms

There is no single universal map from a continuous model to a discrete one. The mechanism depends on what is being discretized.

The most direct finite representation of a continuous function is obtained by sampling it on a grid, but sampling should not be conflated with discretizing an operator. A continuous function

$$f : \mathbb{R}^d \to \mathbb{C} \tag{5.1}$$

WORK IN PROGRESS NOT FOR DISTRIBUTION

may be sampled on a Cartesian grid

$$x_{j,n_j} = a_j + n_j h_j, \qquad n_j \in \mathcal{I}_{N_j}, \tag{5.2}$$

to form

$$X = (x_{\boldsymbol{n}})_{\boldsymbol{n}\in\mathcal{I}_{\boldsymbol{N}}} \in \mathcal{A}_{\boldsymbol{N}}, \qquad x_{\boldsymbol{n}} = f(x_{1,n_1}, \ldots, x_{d,n_d}). \tag{5.3}$$

Sampling alone produces data values; it does not by itself specify a discrete operator.

Sampling supplies point values, whereas quadrature specifies how continuous integrals are replaced by finite sums; the two operations are related but conceptually distinct. An integral

$$\int_\Omega g(\boldsymbol{x})\ \mathrm{d}\boldsymbol{x} \tag{5.4}$$

is replaced by a weighted finite sum

$$\sum_{\boldsymbol{n}} w_{\boldsymbol{n}} g(\boldsymbol{x}_{\boldsymbol{n}}). \tag{5.5}$$

On tensor-product grids, many quadrature rules have product weights

$$w_{\boldsymbol{n}} = \prod_{j=1}^{d} w_{j,n_j}. \tag{5.6}$$

This preserves coordinate separability.

A different route is to discretize the function space itself by projection onto a finite-dimensional trial space. Choose a finite-dimensional trial space

$$V_h = \mathrm{span}\{\Phi_{\boldsymbol{i}}\}. \tag{5.7}$$

Approximate

$$u_h(\boldsymbol{x}) = \sum_{\boldsymbol{i}} c_{\boldsymbol{i}} \Phi_{\boldsymbol{i}}(\boldsymbol{x}). \tag{5.8}$$

Testing a continuous equation against basis/test functions produces a matrix system. If $a(\cdot,\cdot)$ is the relevant bilinear or sesquilinear form, a Petrov–Galerkin matrix has entries

$$A_{q,i} = a(\Phi_i, \Psi_q), \tag{5.9}$$

where $\{\Phi_i\}$ and $\{\Psi_q\}$ are trial and test bases. When the domain, bases, coefficients, and form separate coordinatewise, the resulting matrix inherits Kronecker structure; sums of separated terms lead to sums of Kronecker products.

A complementary discretization is collocation, which enforces the approximation pointwise at selected locations

$$\boldsymbol{x}_{\boldsymbol{n}}. \tag{5.10}$$

WORK IN PROGRESS / NOT FOR DISTRIBUTION

The matrix entries are evaluations such as

$$A_{n,i} = \Phi_i(x_n) \tag{5.11}$$

or

$$A_{n,i} = (\mathcal{L}\Phi_i)(x_n). \tag{5.12}$$

Again, product grids and separable basis functions can produce Kronecker matrices. Boundary treatment is part of the discretization as well: periodic extension, zero extension, truncation, or boundary-adapted bases can change the exact finite-dimensional structure even when the interior operator is unchanged.

6 Fourier Analysis: Separability, the DFT, and the FFT

The tensor structure of Fourier analysis is already visible in the continuous Fourier kernel; see, for example, Plonka et al. [1] for numerical Fourier analysis and multidimensional DFTs. For

$$f : \mathbb{R}^d \to \mathbb{C}, \tag{6.1}$$

$$\widehat{f}(\xi) = \int_{\mathbb{R}^d} f(x) e^{-2\pi i x \cdot \xi} \, \mathrm{d}x. \tag{6.2}$$

Since

$$e^{-2\pi i x \cdot \xi} = \prod_{j=1}^{d} e^{-2\pi i x_j \xi_j}, \tag{6.3}$$

the kernel is separable, giving the tensor-product structure

$$\mathcal{F}_d = \mathcal{F}_1 \otimes \cdots \otimes \mathcal{F}_1$$

on an appropriate tensor-product function space.

6.1 From Periodic Fourier Coefficients to the DFT

For a 2π-periodic function, equivalently

$$f : \mathbb{T}^d \to \mathbb{C}, \qquad \mathbb{T} = \mathbb{R}/(2\pi\mathbb{Z}), \tag{6.4}$$

the Fourier coefficients are

$$c_k(f) = \frac{1}{(2\pi)^d} \int_{[0,2\pi]^d} f(x) e^{-ik \cdot x} \, \mathrm{d}x. \tag{6.5}$$

Choose the uniform grid

WORK IN PROGRESS / NOT FOR DISTRIBUTION

$$x_{j,n_j} = \frac{2\pi n_j}{N_j}, \qquad n_j \in \mathcal{I}_{N_j}, \tag{6.6}$$

and let the discrete frequency index satisfy $\boldsymbol{k} \in \mathcal{I}_{\boldsymbol{N}}$. The sampled tensor

$$X \in \mathcal{A}_{\boldsymbol{N}} \tag{6.7}$$

has entries

$$x_{\boldsymbol{n}} = f\left(\frac{2\pi n_1}{N_1}, \ldots, \frac{2\pi n_d}{N_d}\right). \tag{6.8}$$

Applying the tensor-product rectangle rule gives

$$c_{\boldsymbol{k}}(f) \approx \frac{1}{N_{\text{tot}}} \sum_{\boldsymbol{n}} x_{\boldsymbol{n}} \prod_{j=1}^{d} e^{-2\pi i k_j n_j / N_j}. \tag{6.9}$$

With

$$\omega_{N_j} = e^{-2\pi i / N_j}, \tag{6.10}$$

the unnormalized multidimensional DFT is

$$\widehat{x}_{\boldsymbol{k}} = \sum_{\boldsymbol{n}} x_{\boldsymbol{n}} \prod_{j=1}^{d} \omega_{N_j}^{k_j n_j}, \tag{6.11}$$

so the quadrature relation can be written explicitly as

$$c_{\boldsymbol{k}}(f) \approx \frac{1}{N_{\text{tot}}} \widehat{x}_{\boldsymbol{k}}. \tag{6.12}$$

The approximation sign is important for a general periodic function: the rectangle rule connects continuous Fourier coefficients to the DFT, while sampling identifies frequencies that differ by integer multiples of the grid sizes, producing aliasing. The indices $k_j \in \{0, \ldots, N_j - 1\}$ therefore label frequency classes modulo N_j; negative frequencies may be represented by the corresponding residues. For a trigonometric polynomial whose active frequencies are represented without aliasing on the chosen grid, discrete orthogonality makes the relation exact for those modes.

6.2 Kronecker Representation of the Multidimensional DFT

Define

$$F_{N_j} = [\omega_{N_j}^{k_j n_j}]_{k_j, n_j = 0}^{N_j - 1} \in \mathbb{C}^{N_j \times N_j}. \tag{6.13}$$

Then

$$F_{\boldsymbol{N}} = F_{N_d} \otimes \cdots \otimes F_{N_1} \in \mathbb{C}^{N_{\text{tot}} \times N_{\text{tot}}}, \tag{6.14}$$

WORK IN PROGRESS — NOT FOR DISTRIBUTION

and, because each one-dimensional DFT matrix satisfies $F_{N_j}^{-1} = N_j^{-1} F_{N_j}^*$,

$$F_{\boldsymbol{N}}^{-1} = N_{\mathrm{tot}}^{-1} F_{\boldsymbol{N}}^*. \tag{6.15}$$

Therefore the forward and inverse transforms are

$$\underbrace{\mathrm{vec}(\widehat{X})}_{N_{\mathrm{tot}}\times 1} = \underbrace{(F_{N_d} \otimes \cdots \otimes F_{N_1})}_{N_{\mathrm{tot}}\times N_{\mathrm{tot}}} \underbrace{\mathrm{vec}(X)}_{N_{\mathrm{tot}}\times 1}, \tag{6.16}$$

$$\mathrm{vec}(X) = \frac{1}{N_{\mathrm{tot}}} F_{\boldsymbol{N}}^* \,\mathrm{vec}(\widehat{X}). \tag{6.17}$$

For $d = 2$,

$$X, \widehat{X} \in \mathbb{C}^{N_1 \times N_2}, \tag{6.18}$$

and

$$\mathrm{vec}(\widehat{X}) = (F_{N_2} \otimes F_{N_1})\,\mathrm{vec}(X). \tag{6.19}$$

Under column-major vectorization, the equivalent matrix form is

$$\widehat{X} = F_{N_1} X F_{N_2}^T, \tag{6.20}$$

since

$$\mathrm{vec}(AXB) = (B^T \otimes A)\,\mathrm{vec}(X). \tag{6.21}$$

Thus the transpose on the right factor is the ordinary matrix transpose (not the conjugate transpose): it arises from the identity $\mathrm{vec}(AXB) = (B^T \otimes A)\,\mathrm{vec}(X)$ and from how the second coordinate is stored. Conjugate transposes enter when one forms adjoints or inverse unitary transforms. The reversed order of the Kronecker factors follows directly from the first-coordinate-fastest vectorization convention. For $d = 1$,

$$\widehat{\boldsymbol{x}} = F_N \boldsymbol{x}. \tag{6.22}$$

The Kronecker formula describes the multidimensional DFT algebraically, but it is not how the transform is normally evaluated. Explicitly forming $F_{\boldsymbol{N}} \in \mathbb{C}^{N_{\mathrm{tot}} \times N_{\mathrm{tot}}}$ would discard the main computational advantage of separability. Instead, one applies a one-dimensional transform successively along each coordinate direction:

$$X \xrightarrow{F_{N_1} \text{ along mode } 1} X^{(1)} \xrightarrow{F_{N_2} \text{ along mode } 2} \cdots \xrightarrow{F_{N_d} \text{ along mode } d} \widehat{X}. \tag{6.23}$$

The *fast Fourier transform* (FFT) is not a different transform: it is a family of algorithms for evaluating the DFT by exploiting factorizations of the transform length. A direct length-N DFT costs $O(N^2)$ arithmetic operations, whereas widely used FFT algorithms achieve $O(N \log N)$ complexity (with constants and implementation details depending on the transform length and algorithm). Thus, if each one-dimensional DFT above is evaluated by an FFT, the multidimensional work is approximately

WORK IN PROGRESS / NOT FOR DISTRIBUTION

$$O\left(N_{\text{tot}} \sum_{j=1}^{d} \log N_j\right). \tag{6.24}$$

For $N_1 = \cdots = N_d = N$, this becomes $O(dN^d \log N)$. For an order-d array $X \in \mathcal{A}_{\boldsymbol{N}}$, it is convenient to describe these coordinatewise transforms by a *mode product*. If $A \in \mathbb{C}^{M \times N_j}$, then

$$Y = X \times_j A \in \mathbb{C}^{N_1 \times \cdots \times N_{j-1} \times M \times N_{j+1} \times \cdots \times N_d} \tag{6.25}$$

means that A acts on every mode-j fiber. In components,

$$y_{n_1,\ldots,n_{j-1},m,n_{j+1},\ldots,n_d} = \sum_{r=0}^{N_j-1} A_{m+1,r+1}\, x_{n_1,\ldots,n_{j-1},r,n_{j+1},\ldots,n_d}, \qquad 0 \le m \le M-1. \tag{6.26}$$

Thus the multidimensional DFT can be written as the sequence of mode products

$$\widehat{X} = X \times_1 F_{N_1} \times_2 F_{N_2} \cdots \times_d F_{N_d}, \tag{6.27}$$

under the mode-product convention defined above. This notation describes the transform directly on the multidimensional array; after vectorization, the same coordinatewise action is represented by the Kronecker matrix derived above. Computational implementations replace each dense modewise DFT by its FFT realization. The reason for successive one-dimensional FFTs is therefore both structural and algorithmic: separability makes the coordinate transforms independent, and the FFT applies each of them without assembling the full Kronecker matrix.

7 Wavelets: Multiresolution, Tensor Products, and the FWT

For wavelets, the continuous–discrete connection begins with a multiresolution analysis and its refinement equation rather than with a quadrature rule; a standard reference is Mallat [2]. Let $\{V_s\}_{s\in\mathbb{Z}}$ be a multiresolution analysis of $L^2(\mathbb{R})$ with scaling function ϕ. The refinement equation is

$$\phi(x) = \sqrt{2} \sum_{n\in\mathbb{Z}} h[n]\phi(2x-n). \tag{7.1}$$

An associated wavelet can be written

$$\psi(x) = \sqrt{2} \sum_{n\in\mathbb{Z}} g[n]\phi(2x-n). \tag{7.2}$$

The discrete sequences h and g are coefficients in the refinement relations; they are not obtained merely by sampling ϕ and ψ.

If $a_s[k]$ denotes approximation coefficients at one resolution level, then the refinement relations yield analysis filtering followed by downsampling. Depending on whether the scale index increases toward finer or coarser resolution, the next coarser coefficients may be denoted a_{s-1} or a_{s+1}. With one common real-valued convention,

$$a_{\text{coarse}}[n] = \sum_k h[k-2n]\, a_{\text{fine}}[k], \qquad d_{\text{coarse}}[n] = \sum_k g[k-2n]\, a_{\text{fine}}[k]. \tag{7.3}$$

WORK IN PROGRESS / NOT FOR DISTRIBUTION

Index reversal and complex conjugation depend on whether the filters are defined as correlation or convolution filters; the structural point is the same: one-dimensional filtering is followed by decimation by two. Hence



The one-dimensional multiresolution construction extends to several dimensions by tensor products. Let

$$\theta^0 = \phi, \qquad \theta^1 = \psi. \tag{7.4}$$

For the standard isotropic tensor construction at scale s, define

$$\mathcal{V}_s^{(d)} := V_s^{\otimes d} = \underbrace{V_s \otimes \cdots \otimes V_s}_{d \text{ factors}}. \tag{7.5}$$

and

$$\Psi^{\boldsymbol{\varepsilon}}(\boldsymbol{x}) = \prod_{\ell=1}^{d} \theta^{\varepsilon_\ell}(x_\ell), \qquad \boldsymbol{\varepsilon} \in \{0,1\}^d.$$

There are 2^d tensor combinations: one approximation component and $2^d - 1$ detail components.

With

$$q^0 = h, \qquad q^1 = g, \tag{7.6}$$

the discrete separable filters are

$$q^{\boldsymbol{\varepsilon}} = q^{\varepsilon_d} \otimes \cdots \otimes q^{\varepsilon_1}. \tag{7.7}$$

For

$$X \in \mathcal{A}_{\boldsymbol{N}}, \tag{7.8}$$

one ideal critically sampled level produces 2^d subbands of size

$$\frac{N_1}{2} \times \cdots \times \frac{N_d}{2} \tag{7.9}$$

when all N_j are even.

For $d = 2$, the continuous tensor factors are

$$\phi \otimes \phi, \quad \phi \otimes \psi, \quad \psi \otimes \phi, \quad \psi \otimes \psi, \tag{7.10}$$

and their discrete filter counterparts are

$$h \otimes h, \quad h \otimes g, \quad g \otimes h, \quad g \otimes g. \tag{7.11}$$

WORK IN PROGRESS / NOT FOR DISTRIBUTION

7.1 Wavelet Matrix Representation versus Filter-Bank Implementation

A finite, critically sampled separable discrete wavelet transform also admits a square matrix representation. Assume that boundary handling and the number of decomposition levels are chosen so that the transform preserves the total number of coefficients. Let

$$W_j \in \mathbb{C}^{N_j \times N_j}, \qquad j = 1, \dots, d, \tag{7.12}$$

denote the resulting one-dimensional analysis transform along coordinate direction j. The boundary convention and coefficient ordering are incorporated into W_j. For an orthonormal discrete wavelet transform, W_j may be chosen unitary (orthogonal in the real case); for a biorthogonal construction, the corresponding analysis and synthesis matrices are distinct.

For

$$X, Y \in \mathcal{A}_{\boldsymbol{N}}, \tag{7.13}$$

the multidimensional separable wavelet transform is again a coordinatewise tensor-product action. Under the first-coordinate-fastest vectorization convention used throughout this article, its matrix–vector representation is

$$\operatorname{vec}(Y) = (W_d \otimes \cdots \otimes W_1) \operatorname{vec}(X). \tag{7.14}$$

Accordingly,

$$W_{\boldsymbol{N}} = W_d \otimes \cdots \otimes W_1 \in \mathbb{C}^{N_{\mathrm{tot}} \times N_{\mathrm{tot}}}. \tag{7.15}$$

This Kronecker formula is a coordinate representation of the underlying coordinatewise wavelet transform, rather than its definition.

In two dimensions, the same coordinatewise wavelet action has the direct matrix representation

$$Y = W_1 X W_2^T, \tag{7.16}$$

whereas the stated vectorization convention gives

$$\operatorname{vec}(Y) = (W_2 \otimes W_1) \operatorname{vec}(X). \tag{7.17}$$

Thus $Y = W_1 X W_2^T$ and $\operatorname{vec}(Y) = (W_2 \otimes W_1) \operatorname{vec}(X)$ are two parallel representations of the same two-dimensional coordinatewise wavelet action. The transpose again reflects array orientation, not inversion. If the transform is unitary, synthesis uses W_j^*; in the general invertible or biorthogonal case it uses the appropriate synthesis matrix rather than W_j^T. In particular, the direct matrix formula is not derived from vectorization.

It is essential to distinguish the *wavelet matrix representation* from the *filter-bank implementation*. The matrices W_j record the linear maps effected by the one-dimensional discrete wavelet transforms. A fast DWT normally realizes these maps through local analysis filters, downsampling, boundary handling, and multilevel recursion, without explicitly assembling either the matrices W_j or the full Kronecker matrix $W_d \otimes \cdots \otimes W_1$. The matrix representation is useful for algebraic analysis and for making tensor-product structure explicit, whereas the filter-bank formulation is the computational realization used in practice.

WORK IN PROGRESS — NOT FOR DISTRIBUTION

The tensor products describe the multidimensional wavelet filters, but they need not be formed as full multidimensional arrays or matrices. A separable DWT is implemented by one-dimensional filtering and downsampling along one coordinate at a time. In two dimensions, for example, one first applies the low-pass and high-pass filters in one coordinate direction and downsamples, and then applies the same pair of filters in the other direction. This produces the four subbands

$$LL, \qquad LH, \qquad HL, \qquad HH, \tag{7.18}$$

corresponding to

$$h \otimes h, \qquad h \otimes g, \qquad g \otimes h, \qquad g \otimes g. \tag{7.19}$$

In d dimensions the same coordinatewise procedure produces the 2^d subbands indexed by $\boldsymbol{\varepsilon} \in \{0,1\}^d$. This recursive filtering/downsampling algorithm is commonly called the *fast wavelet transform* (FWT), or Mallat filter-bank algorithm in the standard orthogonal setting. The name refers to a fast algorithm for the discrete wavelet transform, just as FFT refers to a fast algorithm for the DFT.

For a one-dimensional signal of length N and compactly supported filters of fixed length, the first level processes $O(N)$ coefficients, the next approximation level $O(N/2)$, then $O(N/4)$, and so forth. Hence

$$N + \frac{N}{2} + \frac{N}{4} + \cdots = O(N), \tag{7.20}$$

so a full critically sampled one-dimensional FWT has linear complexity for fixed filters. For $X \in \mathcal{A}_{\boldsymbol{N}}$, a separable multidimensional FWT applies the one-dimensional filter bank successively along the d modes. If the dimension d and filter lengths are fixed, and a standard separable critically sampled decomposition is used, the total work is

$$O(N_{\mathrm{tot}}) \tag{7.21}$$

up to constants depending on dimension, filters, boundary handling, and the chosen decomposition scheme. In two dimensions this produces the familiar LL, LH, HL, HH bands at each level; in d dimensions there are 2^d tensor filter combinations per level.

Thus the tensor-product description determines which one-dimensional filters are combined, the wavelet matrix representation records the resulting linear map, and the FWT exploits separability, locality, and decimation to apply that map without assembling the full $N_{\mathrm{tot}} \times N_{\mathrm{tot}}$ wavelet matrix. The asymptotic comparison $O(N)$ for a fixed-filter one-dimensional FWT versus $O(N \log N)$ for a one-dimensional FFT concerns algorithmic work, not a claim that one transform is universally faster in wall-clock time.

8 B-Splines: Tensor-Product Approximation Spaces

B-spline approximation begins with a one-dimensional spline basis and extends to Cartesian product domains by tensor products; see de Boor [3] for the univariate B-spline foundations. Let

$$\{B_{i,p}\}_{i=1}^{n} \tag{8.1}$$

WORK IN PROGRESS NOT FOR DISTRIBUTION

be a one-dimensional B-spline basis of degree p on an interval $\Omega \subset \mathbb{R}$. A spline approximation has the form

$$u_h(x) = \sum_{i=1}^{n} c_i B_{i,p}(x). \tag{8.2}$$

Thus the finite-dimensional space is

$$S_h = \operatorname{span}\{B_{i,p}\}_{i=1}^{n}, \qquad \dim S_h = n. \tag{8.3}$$

For each coordinate direction j, let

$$S_h^{(j)} = \operatorname{span}\{B_{i_j,p_j}^{(j)}\}_{i_j=1}^{n_j}, \qquad \dim S_h^{(j)} = n_j. \tag{8.4}$$

On the Cartesian product domain

$$\Omega = \Omega_1 \times \cdots \times \Omega_d, \tag{8.5}$$

define

$$\boxed{\mathcal{S}_h = S_h^{(1)} \otimes \cdots \otimes S_h^{(d)}.}$$

Its basis functions are

$$\boxed{B_{\boldsymbol{i}}(\boldsymbol{x}) = \prod_{j=1}^{d} B_{i_j,p_j}^{(j)}(x_j),}$$

where

$$\boldsymbol{i} = (i_1, \ldots, i_d), \qquad 1 \le i_j \le n_j. \tag{8.6}$$

The number of basis functions is

$$n_{\text{tot}} = \prod_{j=1}^{d} n_j. \tag{8.7}$$

The tensor-product basis becomes especially useful when the continuous problem is projected onto the spline space, because basis separability can pass directly to the discrete matrices. Consider the mass matrix

$$M_{\boldsymbol{i},\boldsymbol{\ell}} = \int_{\Omega} B_{\boldsymbol{i}}(\boldsymbol{x}) B_{\boldsymbol{\ell}}(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}. \tag{8.8}$$

Because both the domain and basis separate,

WORK IN PROGRESS — NOT FOR DISTRIBUTION

$$M_{\boldsymbol{i},\boldsymbol{\ell}} = \prod_{j=1}^{d} \int_{\Omega_j} B^{(j)}_{i_j,p_j}(x_j) B^{(j)}_{\ell_j,p_j}(x_j)\, \mathrm{d}x_j. \tag{8.9}$$

Define

$$M_j \in \mathbb{R}^{n_j \times n_j}, \tag{8.10}$$

with

$$(M_j)_{i_j,\ell_j} = \int_{\Omega_j} B^{(j)}_{i_j,p_j} B^{(j)}_{\ell_j,p_j}\, \mathrm{d}x_j. \tag{8.11}$$

Then

$$M = M_d \otimes \cdots \otimes M_1 \in \mathbb{R}^{n_{\mathrm{tot}} \times n_{\mathrm{tot}}}.$$

For differential operators, the matrix need not be a single Kronecker product. For example, on a Cartesian domain with constant coefficients, a two-dimensional Laplace-type stiffness matrix typically has a Kronecker-sum form

$$A = K_2 \otimes M_1 + M_2 \otimes K_1,$$

where M_j and K_j are one-dimensional mass and stiffness matrices. For the scalar Laplacian with the usual H^1 bilinear form, for example,

$$(K_j)_{i_j,\ell_j} = \int_{\Omega_j} (B^{(j)}_{i_j,p_j})'(x_j) (B^{(j)}_{\ell_j,p_j})'(x_j)\, \mathrm{d}x_j. \tag{8.12}$$

In d dimensions, define

$$A_j^{(q)} = \begin{cases} K_j, & j = q, \\ M_j, & j \neq q. \end{cases} \tag{8.13}$$

Then the corresponding Cartesian-product stiffness matrix is

$$A = \sum_{q=1}^{d} A_d^{(q)} \otimes \cdots \otimes A_1^{(q)}. \tag{8.14}$$

The two-dimensional formula above is the case $d = 2$. Thus structure-aware discretization can mean a *sum* of Kronecker products, not only one product.

Exact factorization requires that the geometry, coefficients, integration rule, and boundary treatment preserve the relevant coordinate separation. Curved or nonseparable mapped geometries and variable coefficients can destroy the exact single-term structure, although low-rank or sum-of-Kronecker approximations may still be effective.

The B-spline chain is therefore

WORK IN PROGRESS - NOT FOR DISTRIBUTION



The Kronecker structure is also an algorithmic opportunity. One normally does not assemble a dense $n_{\text{tot}} \times n_{\text{tot}}$ tensor matrix merely because a formula such as

$$M = M_d \otimes \cdots \otimes M_1 \tag{8.15}$$

is available. Instead, its action on a coefficient tensor is evaluated by successive one-dimensional mode products. Equivalently, after vectorization,

$$(M_d \otimes \cdots \otimes M_1)\,\mathrm{vec}(X) \tag{8.16}$$

is computed by applying the factors $M_1, \ldots, M_d$ along their respective coordinate directions. For B-splines this is especially useful because the one-dimensional mass and stiffness matrices are banded as a consequence of local support. Hence tensor-product structure and sparsity can be exploited simultaneously.

The same principle applies to Kronecker sums. For example,

$$A = K_2 \otimes M_1 + M_2 \otimes K_1 \tag{8.17}$$

is applied as the sum of two structured tensor contractions rather than as one assembled dense matrix. In higher dimensions, Laplace-type operators lead to a sum of d analogous terms. Matrix-free application is particularly useful in iterative solvers: it reduces storage, preserves the one-dimensional banded factors, and avoids constructing a matrix whose dimension grows as the product of all coordinate dimensions. Assembly may still be appropriate for small problems or for sparse direct methods, but the tensor representation explains how a high-dimensional discretization can be applied through lower-dimensional building blocks.

9 Radial Basis Functions: Radiality versus Separability

For radial basis functions, an important distinction must be made at the outset: radiality does not in general imply coordinate separability; see Wendland [4] for the general theory of scattered-data approximation and radial kernels. An RBF centered at

$$\boldsymbol{c} \in \mathbb{R}^d \tag{9.1}$$

has the form

$$\Phi_{\boldsymbol{c}}(\boldsymbol{x}) = \varphi(\|\boldsymbol{x} - \boldsymbol{c}\|). \tag{9.2}$$

Radiality by itself does *not* imply coordinate separability. In general,

$$\varphi\left(\sqrt{\sum_{j=1}^{d}(x_j - c_j)^2}\right) \neq \prod_{j=1}^{d} \varphi_j(x_j - c_j). \tag{9.3}$$

Therefore a general RBF interpolation matrix should not be claimed to be a Kronecker product merely because the kernel is radial.

WORK IN PROGRESS — NOT FOR DISTRIBUTION

The Gaussian kernel is an important special case in which radiality and coordinate separability coexist. For the Gaussian

$$\Phi_{\boldsymbol{c}}(\boldsymbol{x}) = e^{-\varepsilon^2\|\boldsymbol{x}-\boldsymbol{c}\|_2^2}, \tag{9.4}$$

we have

$$\|\boldsymbol{x}-\boldsymbol{c}\|_2^2 = \sum_{j=1}^{d}(x_j - c_j)^2, \tag{9.5}$$

and hence

$$e^{-\varepsilon^2\|\boldsymbol{x}-\boldsymbol{c}\|_2^2} = \prod_{j=1}^{d} e^{-\varepsilon^2(x_j-c_j)^2}.$$

Thus the isotropic Gaussian is both radial and coordinate-separable. More generally, an anisotropic Gaussian $\exp(-\sum_j \varepsilon_j^2(x_j - c_j)^2)$ remains coordinate-separable, although it is not radial in the ordinary Euclidean norm unless the directional scales coincide.

Suppose both the evaluation points and the centers are Cartesian products:

$$\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d, \qquad \mathcal{C} = \mathcal{C}_1 \times \cdots \times \mathcal{C}_d, \tag{9.6}$$

with $|\mathcal{X}_j| = P_j$, $|\mathcal{C}_j| = N_j$, and $P_{\text{tot}} = \prod_j P_j$. Define the one-dimensional kernel matrices

$$G_j \in \mathbb{R}^{P_j \times N_j}, \tag{9.7}$$

with entries

$$(G_j)_{r,s} = e^{-\varepsilon^2(x_{j,r}-c_{j,s})^2}. \tag{9.8}$$

Then, under compatible first-coordinate-fastest orderings,

$$G = G_d \otimes \cdots \otimes G_1 \in \mathbb{R}^{P_{\text{tot}} \times N_{\text{tot}}}.$$

For interpolation on the same Cartesian-product point set, $\mathcal{X}_j = \mathcal{C}_j$ and hence $P_j = N_j$, so the full kernel matrix is square.

Two conditions are therefore essential:



Without them, an RBF matrix generally has no exact single-Kronecker representation.

Exact rank-one separation is not essential; kernels with finite or approximate separation rank lead to sums of Kronecker products. Even when a kernel is not exactly separable, it may admit an approximation

WORK IN PROGRESS / NOT FOR DISTRIBUTION

$$K(\boldsymbol{x},\boldsymbol{y}) \approx \sum_{r=1}^{R} \prod_{j=1}^{d} K_j^{(r)}(x_j, y_j). \tag{9.9}$$

On a Cartesian grid this gives

$$A \approx \sum_{r=1}^{R} A_d^{(r)} \otimes \cdots \otimes A_1^{(r)}.$$

This extends the separability principle from exact rank-one structure to low separation rank.

For computation, the distinction between an exact separable Gaussian and a general RBF is decisive. On Cartesian-product point sets, the exact Gaussian factorization

$$G = G_d \otimes \cdots \otimes G_1 \tag{9.10}$$

allows a matrix–vector product to be evaluated by successive applications of the one-dimensional kernel matrices G_j, without forming the full $P_{\text{tot}} \times N_{\text{tot}}$ matrix. In the square interpolation case, if the factors are nonsingular, a linear system with a single exact Kronecker product can likewise be reduced to successive one-dimensional solves, since

$$G^{-1} = G_d^{-1} \otimes \cdots \otimes G_1^{-1}. \tag{9.11}$$

This reduces storage and exposes smaller linear-algebra problems, although the conditioning of the one-dimensional RBF matrices remains an independent numerical issue.

For a nonseparable kernel, or for non-Cartesian point sets, this exact coordinatewise algorithm is generally unavailable. If a low-separation-rank approximation

$$A \approx \sum_{r=1}^{R} A_d^{(r)} \otimes \cdots \otimes A_1^{(r)} \tag{9.12}$$

has been constructed, each term can still be applied by successive mode products and the R results can be accumulated. Thus the practical reason for seeking separable or low-rank kernel representations is not merely a more compact formula: it replaces one high-dimensional dense operation by a small number of lower-dimensional operations. When no such representation is available, other RBF acceleration techniques are required and should not be attributed to tensor-product separability.

10 Convolution: Transform Representations and Structured Matrices

For convolution, the starting structural property is translation invariance. For $f, \kappa : \mathbb{R}^d \to \mathbb{C}$ for which the integral is well defined, let

$$(\mathcal{K}f)(\boldsymbol{x}) = (\kappa * f)(\boldsymbol{x}) = \int_{\mathbb{R}^d} \kappa(\boldsymbol{x}-\boldsymbol{y}) f(\boldsymbol{y})\, \mathrm{d}\boldsymbol{y}. \tag{10.1}$$

WORK IN PROGRESS — NOT FOR DISTRIBUTION

Convolution acts on functions (or, after discretization, on vectors and multidimensional arrays); it is not the same operation as ordinary matrix multiplication or the Hadamard product. Its importance here is that convolution kernels depend only on the difference $\boldsymbol{x} - \boldsymbol{y}$. Under the standard hypotheses for translation-invariant continuous linear operators, this difference-kernel structure is the continuous origin of convolution.

The Fourier transform converts convolution into pointwise multiplication. With the Fourier convention of Section 6, the convolution theorem states

$$\mathcal{F}(\kappa * f)(\boldsymbol{\xi}) = \widehat{\kappa}(\boldsymbol{\xi})\, \widehat{f}(\boldsymbol{\xi})$$

and therefore

$$\mathcal{K} f = \mathcal{F}^{-1}(\widehat{\kappa}\, \mathcal{F} f)\,.$$

Thus, on function spaces for which the Fourier transform and multiplier are well defined, a translation-invariant convolution operator is transformed into a multiplication operator: each frequency is multiplied by the scalar Fourier multiplier $\widehat{\kappa}(\boldsymbol{\xi})$. In the periodic/discrete setting this becomes literal diagonalization by the Fourier basis. For a vector-valued field $f : \mathbb{R}^d \to \mathbb{C}^{c_{\text{in}}}$ with c_{out} output channels, the multiplier becomes a matrix

$$R(\boldsymbol{\xi}) \in \mathbb{C}^{c_{\text{out}} \times c_{\text{in}}}, \qquad \widehat{(\mathcal{K} f)}(\boldsymbol{\xi}) = R(\boldsymbol{\xi}) \widehat{f}(\boldsymbol{\xi}). \tag{10.2}$$

After frequency discretization, scalar pointwise multiplication becomes the *Hadamard product*. For two arrays A and B of the same dimensions,

$$(A \odot B)_{\boldsymbol{k}} = A_{\boldsymbol{k}} B_{\boldsymbol{k}}. \tag{10.3}$$

Thus, if $\widehat{K}$ and $\widehat{X}$ are same-size scalar Fourier-coefficient arrays,

$$\widehat{Y} = \widehat{K} \odot \widehat{X}. \tag{10.4}$$

With multiple channels, the operation is instead a small matrix–vector multiplication at each frequency index. This distinction explains why Hadamard products appear naturally in scalar spectral convolution while matrix-valued multipliers are needed for channel mixing.

Wavelets provide a different representation of the same operator. Let $\mathcal{W}$ denote an invertible wavelet transform between the physical/function representation and its wavelet-coordinate space. Whenever the compositions below are well defined, define the wavelet-coordinate representation of $\mathcal{K}$ by

$$A_W = \mathcal{W} \mathcal{K} \mathcal{W}^{-1}. \tag{10.5}$$

Then, exactly,

$$\mathcal{K} = \mathcal{W}^{-1} A_W \mathcal{W}, \qquad \mathcal{K} f = \mathcal{W}^{-1}(A_W\, \mathcal{W} f)\,.$$

WORK IN PROGRESS — NOT FOR DISTRIBUTION

This formula is the wavelet analogue of expressing an operator in transformed coordinates, but it must not be confused with the Fourier convolution theorem. Fourier characters are generalized eigenfunctions of translation-invariant convolution on $\mathbb{R}^d$ (and ordinary Fourier basis functions in the periodic setting), so the Fourier transform reduces convolution to multiplication. A general wavelet basis does *not* exactly diagonalize an arbitrary convolution operator. Consequently A_W is generally not diagonal and its action is not, in general, a simple Hadamard product. Nevertheless, because wavelets are localized in both position and scale, many operators have sparse, compressible, block-local, or approximately diagonal representations in wavelet coordinates. This localization is the reason wavelet-domain operator models can parameterize interactions across scale and space without claiming the exact diagonalization enjoyed by Fourier convolution.

The two transformed representations can therefore be summarized as

$$\boxed{\mathcal{K}_{\mathrm{conv}} = \mathcal{F}^{-1} M_{\widehat{\kappa}} \mathcal{F}, \qquad \mathcal{K} = \mathcal{W}^{-1} A_W \mathcal{W},}$$

where $M_{\widehat{\kappa}}$ is the diagonal multiplication operator associated with the Fourier multiplier, whereas A_W is the generally non-diagonal wavelet-coordinate operator. In learned operator models one may replace or parameterize these transformed-domain operators by trainable maps, but the mathematical distinction between exact Fourier diagonalization and a general wavelet-basis representation remains essential.

If, in addition, the convolution kernel is coordinate-separable,

$$\kappa(\boldsymbol{z}) = \prod_{j=1}^{d} \kappa_j(z_j), \tag{10.6}$$

then

$$\boxed{\mathcal{K} = \mathcal{K}_d \otimes \cdots \otimes \mathcal{K}_1.}$$

Here tensor products express independence across coordinate directions. They play a different role from the Hadamard product: $\otimes$ builds a multidimensional operator from coordinate operators, while $\odot$ multiplies same-shaped coefficients entry by entry.

Sampling the convolution operator on a Cartesian grid and approximating the integral by product quadrature produces a discrete convolution. Choose

$$x_{j,n_j} = n_j h_j, \qquad h_j > 0, \tag{10.7}$$

and let

$$x_{\boldsymbol{m}} = f(m_1 h_1, \ldots, m_d h_d). \tag{10.8}$$

A product quadrature rule gives

$$(\mathcal{K}f)(n_1 h_1, \ldots, n_d h_d) \approx \left(\prod_{j=1}^{d} h_j \right) \sum_{\boldsymbol{m}} \kappa(h_1(n_1 - m_1), \ldots, h_d(n_d - m_d)) x_{\boldsymbol{m}}. \tag{10.9}$$

WORK IN PROGRESS / NOT FOR DISTRIBUTION

Defining

$$k_{\boldsymbol{r}} = \left(\prod_{j=1}^{d} h_j \right) \kappa(h_1 r_1, \ldots, h_d r_d) \tag{10.10}$$

yields

$$\boxed{y_{\boldsymbol{n}} \approx \sum_{\boldsymbol{m}} k_{\boldsymbol{n}-\boldsymbol{m}} x_{\boldsymbol{m}}.}$$

Dependence on the multi-index difference $\boldsymbol{n} - \boldsymbol{m}$ produces multilevel Toeplitz structure; tensor properties of such matrices are discussed, for example, by Olshevsky, Oseledets, and Tyrtyshnikov [6], while a broader multilevel Toeplitz framework is developed by Garoni and Serra-Capizzano [7]. For a same-size finite nonperiodic discretization, take $\boldsymbol{n}, \boldsymbol{m} \in \mathcal{I}_{\boldsymbol{N}}$ and interpret samples outside the finite input grid according to zero extension or an equivalent truncation convention. Rows and columns may then be labeled by grid multi-indices:

$$T_{\boldsymbol{N}}(k) \in \mathbb{C}^{N_{\mathrm{tot}} \times N_{\mathrm{tot}}}, \qquad [T_{\boldsymbol{N}}(k)]_{\ell_{\boldsymbol{N}}(\boldsymbol{n})+1,\, \ell_{\boldsymbol{N}}(\boldsymbol{m})+1} = k_{\boldsymbol{n}-\boldsymbol{m}}. \tag{10.11}$$

Thus dependence on the difference of grid indices becomes Toeplitz structure after the multi-indices are linearly ordered. If

$$k_{\boldsymbol{r}} = \prod_{j=1}^{d} k_j(r_j), \tag{10.12}$$

then

$$\boxed{T_{\boldsymbol{N}}(k) = T_{N_d}(k_d) \otimes \cdots \otimes T_{N_1}(k_1).}$$

For $d = 2$ this is a block Toeplitz matrix with Toeplitz blocks (BTTB). More generally, a finite separation-rank kernel

$$k_{\boldsymbol{r}} = \sum_{\ell=1}^{R} \prod_{j=1}^{d} k_j^{(\ell)}(r_j) \tag{10.13}$$

produces a sum of Kronecker products,

$$T_{\boldsymbol{N}}(k) = \sum_{\ell=1}^{R} T_{N_d}(k_d^{(\ell)}) \otimes \cdots \otimes T_{N_1}(k_1^{(\ell)}). \tag{10.14}$$

Boundary conditions determine whether the finite-dimensional convolution matrix is Toeplitz or circulant. Nonperiodic finite convolution gives Toeplitz, BTTB, or multilevel Toeplitz matrices. Periodic convolution gives circulant or multilevel circulant matrices. In the periodic case,

$$C_{\boldsymbol{N}} = F_{\boldsymbol{N}}^{-1} \Lambda F_{\boldsymbol{N}}, \qquad F_{\boldsymbol{N}} = F_{N_d} \otimes \cdots \otimes F_{N_1}, \tag{10.15}$$

WORK IN PROGRESS — NOT FOR DISTRIBUTION

where the matrices F_{N_j} use the unnormalized DFT convention of Section 6. Hence

$$F_{N_j}^{-1} = N_j^{-1} F_{N_j}^*, \qquad F_{\boldsymbol{N}}^{-1} = N_{\mathrm{tot}}^{-1} F_{\boldsymbol{N}}^*. \tag{10.16}$$

For scalar periodic convolution, Λ is diagonal and contains the corresponding discrete Fourier multiplier. A nonperiodic Toeplitz matrix is not generally exactly diagonalized by the DFT, although Toeplitz matrix–vector products can be accelerated by embedding the operator into a larger circulant problem with appropriate padding.

The transformed and structured formulas determine practical algorithms. For periodic scalar convolution,

$$X \xrightarrow{\text{multidimensional FFT}} \widehat{X} \xrightarrow{\odot \widehat{K}} \widehat{Y} \xrightarrow{\text{inverse FFT}} Y. \tag{10.17}$$

For a separable spatial kernel, one may instead apply one-dimensional convolutions successively along the tensor modes. For wavelet-coordinate methods, one applies the multidimensional FWT, acts on the resulting multiscale coefficient tensor by A_W or a structured approximation to it, and applies the inverse FWT. The notation introduced throughout the article has a clear division of labor: convolution describes translation-invariant interaction in physical space; Hadamard multiplication expresses scalar pointwise interaction in a discrete transform domain; tensor products combine independent factors; mode products apply coordinate transforms directly to multidimensional arrays; and Kronecker products represent tensor-product linear maps after vectorization. These operations are related by the continuous–discrete constructions above, but they act on different mathematical objects and are not interchangeable.

11 Summary

The tensor and structured-matrix formulas in this article should be read at two complementary levels. Algebraically, they identify the finite-dimensional representation inherited from the continuous model. Algorithmically, they identify operations that should not be carried out as generic dense matrix multiplication. Fourier transforms use successive one-dimensional FFTs; separable wavelets use coordinatewise filter banks and downsampling; tensor-product B-spline discretizations use sparse one-dimensional factors and matrix-free tensor contractions; separable Gaussian RBF matrices use successive one-dimensional kernel operations or solves; and Toeplitz/circulant operators use convolution structure, often together with FFT acceleration. The common computational principle is to preserve the structure exposed by separability, translation invariance, or local support rather than destroy it by assembling an unnecessarily large unstructured matrix.

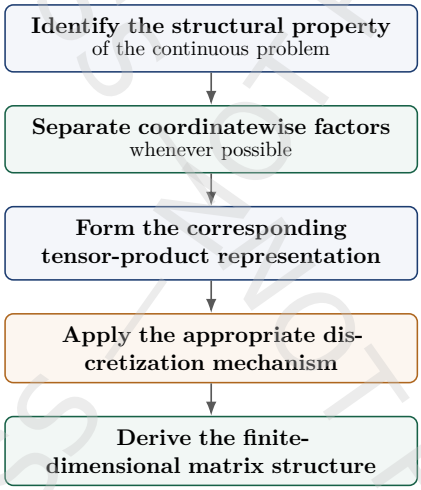
The examples developed above can now be viewed within a single framework, provided that the source of each discrete structure is kept explicit. Separability is the mechanism that leads from coordinatewise continuous objects to tensor-product representations. After finite-dimensional bases and an index ordering are chosen, these tensor-product operators are represented by Kronecker products or, more generally, by sums of Kronecker products. Other structures have different origins: translation invariance produces Toeplitz or circulant matrices, while local support often produces sparsity.

The main examples are summarized below.

WORK IN PROGRESS / NOT FOR DISTRIBUTION

Object	Continuous structure	Discretization mechanism	Discrete structure
Fourier	$e^{-2\pi i \boldsymbol{x}\cdot\boldsymbol{\xi}} = \prod_j e^{-2\pi i x_j \xi_j}$	periodic sampling and quadrature	$F_{N_d} \otimes \cdots \otimes F_{N_1}$
Wavelet	$\mathcal{V}_s^{(d)} = V_s^{\otimes d}$ and tensor-product basis functions	refinement equations, filtering, and downsampling	separable multidimensional filter banks
B-spline	$B_{\boldsymbol{i}}(\boldsymbol{x}) = \prod_j B_{i_j,p_j}^{(j)}(x_j)$	Galerkin projection or collocation	Kronecker products, Kronecker sums, and sparse matrices
Gaussian RBF	$e^{-\varepsilon^2\|\boldsymbol{x}-\boldsymbol{c}\|^2} = \prod_j e^{-\varepsilon^2(x_j-c_j)^2}$	kernel evaluation on Cartesian-product point sets	Kronecker-structured kernel matrix
Convolution	translation invariance $\kappa(\boldsymbol{x}-\boldsymbol{y})$, with optional kernel separability	sampling and quadrature	Toeplitz, BTTB, or multilevel Toeplitz; Kronecker factorization for separable kernels

A useful workflow is therefore



For the separable part of this workflow, the central implication is



This statement is deliberately narrower than saying that every structured matrix is a consequence of separability.

The structural reductions above also have direct computational consequences. Suppose, for simplicity, that

$$N_1 = \cdots = N_d = N, \qquad N_{\text{tot}} = N^d. \tag{11.1}$$

A general dense operator on $\mathbb{C}^{N^d}$ contains

$$N^{2d} \tag{11.2}$$

entries. By contrast, a single tensor-product operator

WORK IN PROGRESS — NOT FOR DISTRIBUTION

$$A = A_d \otimes \cdots \otimes A_1, \qquad A_j \in \mathbb{C}^{N \times N}, \tag{11.3}$$

is specified by d one-dimensional factors, requiring only

$$dN^2 \tag{11.4}$$

entries if those factors are stored as dense matrices.

The more important advantage is computational. If the one-dimensional factors are dense, applying $A_d \otimes \cdots \otimes A_1$ by successive mode products costs $O(dN^{d+1}) = O(dN\, N_{\text{tot}})$ arithmetic operations, rather than the $O(N^{2d})$ work of a generic dense matrix–vector product with the assembled operator. If each factor has a fixed bandwidth b, the cost drops to $O(dbN^d)$. These estimates are leading-order counts and do not include data-movement effects, but they make the structural saving explicit. This viewpoint underlies axis-by-axis Fourier transforms, separable wavelet filter banks, successive one-dimensional convolutions, modewise application of tensor-product spline operators, and structured operations with separable RBF kernel matrices.

If exact separability is unavailable but a low-separation-rank approximation exists,

$$A \approx \sum_{r=1}^{R} A_d^{(r)} \otimes \cdots \otimes A_1^{(r)}, \tag{11.5}$$

then the separation rank R controls the tradeoff between representation cost and approximation accuracy.

For reference, the principal discrete dimensions used throughout the article are

$$\begin{aligned}
&X, \widehat{X}, Y && \in \mathcal{A}_{\boldsymbol{N}} = \mathbb{C}^{\mathcal{I}_{\boldsymbol{N}}},\\
&\operatorname{vec}(X), \operatorname{vec}(\widehat{X}), \operatorname{vec}(Y) && \in \mathbb{C}^{N_{\text{tot}}},\\
&F_{N_j} && \in \mathbb{C}^{N_j \times N_j},\\
&F_{\boldsymbol{N}} && \in \mathbb{C}^{N_{\text{tot}} \times N_{\text{tot}}},\\
&T_{N_j}(k_j) && \in \mathbb{C}^{N_j \times N_j},\\
&T_{\boldsymbol{N}}(k) && \in \mathbb{C}^{N_{\text{tot}} \times N_{\text{tot}}},\\
&M_j, K_j && \in \mathbb{R}^{n_j \times n_j} \quad \text{(B-spline matrices)},\\
&M, A && \in \mathbb{R}^{n_{\text{tot}} \times n_{\text{tot}}}, \quad n_{\text{tot}} = \textstyle\prod_j n_j,\\
&G_j && \in \mathbb{R}^{P_j \times N_j} \quad \text{(Cartesian Gaussian RBF)},\\
&G && \in \mathbb{R}^{P_{\text{tot}} \times N_{\text{tot}}}, \qquad P_{\text{tot}} = \textstyle\prod_j P_j.
\end{aligned}$$

In summary, Fourier transforms, separable wavelets, tensor-product B-splines, separable RBF kernels, and multidimensional convolution are distinct constructions with different continuous origins. What unifies them is a methodology: identify the continuous structure first, represent coordinatewise separability by tensor products when it is present, and only then derive the matrix structure produced by the chosen discretization. This order of reasoning prevents Kronecker, Toeplitz, circulant, sparse, and block structures from being conflated while making their relationships transparent.

WORK IN PROGRESS / NOT FOR DISTRIBUTION

References

[1] G. Plonka, D. Potts, G. Steidl, and M. Tasche, *Numerical Fourier Analysis*, 2nd ed., Birkhäuser, Cham, 2023.

[2] S. Mallat, *A Wavelet Tour of Signal Processing: The Sparse Way*, 3rd ed., Academic Press, 2009.

[3] C. de Boor, *A Practical Guide to Splines*, rev. ed., Springer, New York, 2001.

[4] H. Wendland, *Scattered Data Approximation*, Cambridge University Press, Cambridge, 2005.

[5] C. F. Van Loan, "The ubiquitous Kronecker product," *Journal of Computational and Applied Mathematics*, vol. 123, nos. 1–2, pp. 85–100, 2000.

[6] V. Olshevsky, I. Oseledets, and E. Tyrtyshnikov, "Tensor properties of multilevel Toeplitz and related matrices," *Linear Algebra and its Applications*, vol. 412, no. 1, pp. 1–21, 2006.

[7] C. Garoni and S. Serra-Capizzano, *Generalized Locally Toeplitz Sequences: Theory and Applications, Volume II*, Springer, Cham, 2018.

WORK IN PROGRESS — NOT FOR DISTRIBUTION